

PC 505 AI | BE AIML — Osmania University

Operating Systems

Complete 2-Mark & 6-Mark Exam Answers

Unit I • Unit II • Unit III • Unit IV • Unit V

Reference: Silberschatz, Galvin & Gagne — OS Concepts 9th Ed.

Unit	Topics Covered
Unit I	OS Structure, Process Concepts, Multithreading, Scheduling, Synchronization, Deadlocks
Unit II	Swapping, Contiguous Allocation, Paging, Segmentation, Virtual Memory, Page Replacement, Thrashing
Unit III	File Concepts, Access Methods, File System Implementation, Allocation Methods, I/O Systems
Unit IV	Protection Principles, Access Matrix, Cryptography, Security Threats, Authentication, Firewalling
Unit V	Linux Design, Kernel Modules, Linux Process/Memory/FS, Windows 7 Architecture & Components

Unit I — Introduction, Process Concepts, Scheduling, Synchronization, Deadlocks

OS Structure | Process States | Threads | CPU Scheduling | Critical Section | Semaphores | Deadlock

2-MARK QUESTIONS — Unit I

2 Marks

Q1. Define Operating System. What are its two main roles?

An **Operating System (OS)** is system software that manages hardware resources and provides services to user programs. It acts as an intermediary between users and hardware.

Two main roles: (1) **Resource allocator** — manages CPU, memory, I/O devices, and allocates them to processes. (2) **Control program** — controls execution of user programs to prevent errors and improper use of the computer.

2 Marks

Q2. What is a process? What are the components stored in its PCB?

A **process** is a program in execution. It is an active entity — the program code (text section) plus the current activity (program counter, registers, stack, heap).

Process Control Block (PCB) stores: Process ID (PID), process state, program counter, CPU registers, CPU scheduling information (priority, queue pointers), memory management info (page tables, base/limit registers), I/O status (list of open files), accounting info (CPU used, time limits).

2 Marks

Q3. List and explain the five process states.

State	Description
New	Process is being created (fork() called, PCB allocated)
Ready	In memory, waiting to be assigned to CPU (in ready queue)
Running	Instructions currently being executed by CPU
Waiting	Waiting for I/O completion or event (blocked, not using CPU)
Terminated	Process has finished execution (exit() called)

2 Marks

Q4. What is a thread? State the difference between a process and a thread.

A **thread** is the smallest unit of CPU execution within a process. It is a lightweight process that shares the process's code, data, and OS resources but has its own program counter, stack, and registers.

Property	Process	Thread
Memory	Separate address space	Shares process address space
Creation overhead	Heavy (fork/exec)	Light (pthread_create)
Communication	IPC needed	Direct shared memory
Crash effect	Only this process affected	Can crash whole process

2 Marks

Q5. Define critical section problem. State its three requirements.

The **critical section** is a code segment where a process accesses shared resources (shared variables, files, tables). Only one process should execute in its critical section at a time.

Three requirements: (1) **Mutual exclusion** — if process P_i is in critical section, no other process is. (2) **Progress** — if no process is in critical section, the decision on who enters next cannot be postponed indefinitely. (3) **Bounded waiting** — there is a limit on how many times other processes can enter before a waiting process gets in.

2 Marks

Q6. State the four necessary conditions for a deadlock.

A deadlock exists if and only if all four Coffman conditions hold simultaneously:

(1) **Mutual Exclusion** — resource held non-sharably. (2) **Hold and Wait** — process holds one resource and waits for another. (3) **No Preemption** — resources cannot be forcibly taken away. (4) **Circular Wait** — circular chain $P_1 \rightarrow P_2 \rightarrow \dots \rightarrow P_n \rightarrow P_1$ of waiting processes exists.

2 Marks

Q7. What is context switching? Why is it called overhead?

Context switching is the process of saving the state (context) of the currently running process in its PCB and loading the state of the next process to run. It enables multitasking.

It is called **overhead** because the CPU does NO useful work while switching — it only saves and restores state. If context switching is too frequent (e.g., very small time quantum in RR), a significant fraction of CPU time is wasted on switching rather than running user processes.

6-MARK QUESTIONS — Unit I

6 Marks

Q1. Explain the various OS structures (Monolithic, Layered, Microkernel, Modular) with advantages and disadvantages.

An OS can be designed using several structural approaches. Each trades off simplicity, performance, and maintainability differently.

1. Simple/Monolithic Structure

All OS functions are placed in a single large kernel with no modular structure. All procedures can call any other procedure.

Example: MS-DOS — no well-defined structure. The application programs could access basic I/O routines directly. Early UNIX — kernel contains all features: file system, CPU scheduling, memory management, device drivers.

- **Advantage:** Very fast — direct procedure calls, no message passing overhead
- **Disadvantage:** Hard to modify, debug, maintain. A bug in one module can crash the whole OS

2. Layered Structure

OS divided into layers. Layer 0 = hardware. Layer N = user interface. Each layer uses services only from the layer below it.

Layer 5: User programs Layer 4: I/O operations Layer 3: Operator-process communication Layer 2: Memory management Layer 1: CPU scheduling Layer 0: Hardware

- **Advantage:** Easy debugging — test one layer at a time. Clean interfaces between layers
- **Disadvantage:** Less efficient — request must pass through multiple layers. Hard to define clean layer boundaries

3. Microkernel Structure

Keeps only the most essential functions in the kernel (IPC, basic scheduling, basic memory management). All other services (device drivers, file systems) run as user-space processes.

Example: macOS (Mach microkernel), QNX, Minix 3. Kernel size: only ~10,000 lines of code vs monolithic kernels with millions.

- **Advantage:** Reliable — a device driver crash does not crash the kernel. Easy to extend — add services without modifying kernel
- **Disadvantage:** Performance overhead — services communicate via message passing (slow vs direct calls)

4. Modular Structure (Loadable Kernel Modules)

Core kernel with dynamically loadable modules. Best approach used by modern OS (Linux, Solaris, Windows). Each module implements an interface and can be loaded/unloaded at runtime.

Linux example: lsmod (list modules), insmod (insert), rmmod (remove) Modules for: device drivers, file systems (ext4, NTFS), network protocols (TCP/IP)

- **Advantage:** Best of both — monolithic performance (direct calls) + microkernel flexibility (loadable at runtime)
- **Disadvantage:** Bugs in modules can still crash kernel (not as safe as microkernel)

6 Marks

Q2. Explain CPU Scheduling algorithms: FCFS, SJF, Priority, and Round Robin with Gantt charts and average waiting time.

CPU scheduling selects which process from the ready queue gets the CPU next. Key metrics: **Waiting Time (WT)** = TAT – Burst, **Turnaround Time (TAT)** = Completion – Arrival.

Example data (all arrive at time 0):

Process	Burst Time (ms)	Priority
P1	10	3
P2	1	1
P3	2	3
P4	1	4
P5	5	2

1. FCFS (First Come First Served) — Service order: P1,P2,P3,P4,P5

P1(0-10) | P2(10-11) | P3(11-13) | P4(13-14) | P5(14-19) WT: P1=0, P2=10, P3=11, P4=13, P5=14 → Avg WT = $48/5 = 9.6$ ms

2. SJF (Shortest Job First) — Non-preemptive, order by burst: P2(1),P4(1),P3(2),P5(5),P1(10)

P2(0-1) | P4(1-2) | P3(2-4) | P5(4-9) | P1(9-19) WT: P2=0, P4=1, P3=2, P5=4, P1=9 → Avg WT = $16/5 = 3.2$ ms [OPTIMAL]

3. Non-Preemptive Priority — Lower number = higher priority: P2(1),P5(2),P1(3),P3(3),P4(4)

P2(0-1) | P5(1-6) | P1(6-16) | P3(16-18) | P4(18-19) WT: P2=0, P5=1, P1=6, P3=16, P4=18 → Avg WT = $41/5 = 8.2$ ms

4. Round Robin (q=2ms) — Each process gets 2ms slice:

```
P1(0-2) | P2(2-3) | P3(3-5) | P4(5-6) | P5(6-8) | P1(8-10) | P5(10-12) | P1(12-14) | P5(14-15) | P1(15-17) | P1(17-19) WT: P1=9, P2=2, P3=3, P4=5, P5=9 → Avg WT = 28/5 = 5.6 ms
```

Key comparison: SJF minimises average waiting time (proven optimal). RR is fairest for interactive systems. FCFS is simplest but gives convoy effect (short jobs wait behind long ones).

6 Marks

Q3. Explain Process Synchronization — Critical Section Problem, Semaphores, and Classic Problems.

When multiple processes share data, **race conditions** can produce incorrect results. Synchronization ensures orderly access to shared resources.

Peterson's Solution — software solution for 2 processes:

```
Shared: int turn; boolean flag[2]; Process Pi: flag[i] = true; turn = i; while (flag[j] && turn == j); // busy wait // CRITICAL SECTION flag[i] = false;
```

Semaphores — synchronization tool with two atomic operations:

```
wait(S): while (S <= 0) do nothing; S = S-1; // P operation signal(S): S = S+1; // V operation Binary semaphore (mutex): S = 0 or 1 Counting semaphore: S = any non-negative integer
```

Classic Synchronization Problems:

Problem	Shared resource	Solution approach
Producer-Consumer	Bounded buffer (circular queue)	Mutex + full + empty semaphores
Readers-Writers	Shared database — multiple readers OR one writer	Reader count + mutex + write lock
Dining Philosophers	5 forks for 5 philosophers — need 2 forks to eat	Asymmetric fork pickup OR monitor solution

Producer-Consumer solution example:

```
Semaphore mutex=1, empty=n, full=0; Producer: Consumer: wait(empty) wait(full) wait(mutex) wait(mutex) add item remove item signal(mutex) signal(mutex) signal(full) signal(empty)
```

6 Marks

Q4. Explain Deadlock — Conditions, Prevention, Avoidance (Banker's Algorithm), Detection and Recovery.

A **deadlock** is a state where a set of processes are each waiting for a resource held by another in the set — circular waiting with no way out.

1. Prevention — Negate one of the four conditions:

Condition to negate	Method	Drawback
Mutual exclusion	Make resources shareable	Not always possible (printers, etc.)
Hold and wait	Request ALL resources at start or release before requesting new	Low resource utilization; starvation
No preemption	If waiting, release all held resources	May cause loss of partial computation

Circular wait	Total ordering on resource types — always request in increasing order	Not always convenient
----------------------	--	-----------------------

2. Avoidance — Banker's Algorithm

OS maintains safe state — only grants resource requests that keep system in a safe state (a safe sequence exists).

```
Need[i][j] = Max[i][j] - Allocation[i][j]
Safety algorithm: Work = Available
Find process i where Finish[i]=false AND Need[i] <= Work
Work = Work + Allocation[i];
Finish[i] = true
Repeat until all Finish[i]=true (SAFE) or no such i (UNSAFE)
```

3. Detection — Resource allocation graph (RAG). Cycle in graph → deadlock possible. Wait-for graph for multiple instances.

4. Recovery — (a) Process termination: kill all deadlocked processes, or kill one at a time. (b) Resource preemption: select victim, rollback, prevent starvation.

Unit II — Memory Management Strategies

Swapping | Contiguous Allocation | Paging | Segmentation | Virtual Memory | Page Replacement | Thrashing

2-MARK QUESTIONS — Unit II

2 Marks

Q1. What is swapping in memory management?

Swapping is a memory management technique where a process is temporarily moved out of main memory to a backing store (swap space on disk) and brought back into memory later for continued execution.

When process P1 is idle or waiting for I/O, the OS **swaps out** P1 to disk and **swaps in** another process P2. When P1 is ready again, it is swapped back in. This allows more processes to share memory than would physically fit simultaneously.

2 Marks

Q2. What is paging? Define page, frame, and page table.

Paging is a memory management scheme that divides both logical address space and physical memory into equal fixed-size blocks, eliminating external fragmentation.

Page: Fixed-size block of logical address space. **Frame:** Fixed-size block of physical memory (same size as page, typically 4KB). **Page table:** OS data structure mapping each logical page number to the physical frame number where it is stored.

2 Marks

Q3. What is a page fault? What happens when a page fault occurs?

A **page fault** occurs when a process tries to access a page that is not currently loaded in physical memory (the valid bit in the page table entry is 0 — the page is on disk).

Steps when page fault occurs: (1) OS traps page fault. (2) Find location of desired page on disk. (3) Find a free frame (or use page replacement algorithm). (4) Load the page from disk into the frame. (5) Update page table entry (set valid bit to 1). (6) Restart the instruction that caused the fault.

2 Marks

Q4. Define thrashing. What causes it?

Thrashing occurs when a process does not have enough frames and spends more time paging (swapping pages in/out) than executing. The system is extremely busy swapping but accomplishes very little useful work.

Cause: If the working set of a process (set of pages actively used) is larger than available frames, the process keeps generating page faults. As more processes are added, each gets fewer frames, causing all to thrash. CPU utilization drops; OS mistakenly loads more processes, making it worse.

2 Marks

Q5. Differentiate internal fragmentation and external fragmentation.

Property	Internal Fragmentation	External Fragmentation
Definition	Wasted space INSIDE an allocated partition	Free memory blocks scattered between allocated partitions
Cause	Process smaller than fixed partition	Variable-size partitions; after many allocate/free cycles
Solution	Smaller partition sizes	Compaction; paging (eliminates completely)
Occurs in	Fixed partitioning, paging	Variable partitioning, segmentation

2 Marks

Q6. What is demand paging? State its advantage over pure paging.

Demand paging loads pages into memory only when they are demanded (accessed) by the process — NOT all at once when the process starts. Pages are loaded on demand when a page fault occurs.

Advantage: (1) Less memory used — only active pages loaded. (2) Faster process startup — no need to load entire program first. (3) More processes can run simultaneously. (4) Enables programs larger than physical memory to execute (virtual memory).

2 Marks

Q7. What is the difference between logical address and physical address?

Property	Logical Address	Physical Address
Definition	Address generated by CPU during execution	Actual address in physical memory (RAM)
Also called	Virtual address	Memory address
Generated by	CPU (during instruction execution)	Memory Management Unit (MMU) hardware
Visible to	User program	OS and hardware
Conversion	MMU translates logical to physical via page table	Direct access to RAM chip

6-MARK QUESTIONS — Unit II

6 Marks

Q1. Explain Contiguous Memory Allocation — Fixed and Variable Partitioning with First Fit, Best Fit, and Worst Fit strategies.

Contiguous allocation requires each process to be stored in a single contiguous block of memory. Two approaches exist:

Fixed Partitioning

Memory divided into fixed-size partitions at system startup. Each partition holds exactly one process.

- Simple to implement — OS maintains a table of partition status (free/occupied)
- **Internal fragmentation:** Process 60KB in 100KB partition → 40KB wasted inside partition
- Degree of multiprogramming limited by number of fixed partitions

Variable Partitioning

Partitions created dynamically to match exact process size. OS maintains list of free holes (free memory blocks) and uses a placement algorithm:

Memory state: [100K free] | [240K occupied-P1] | [300K free] | [200K occupied-P2] | [320K free]
 Process requests 250K — which hole to use?

Strategy	Algorithm	250K request goes to	Leftover
First Fit	Allocate FIRST hole large enough	300K hole (first fit)	50K fragment
Best Fit	Allocate SMALLEST hole large enough	300K hole (best fit here)	50K fragment
Worst Fit	Allocate LARGEST hole	320K hole	70K fragment (largest leftover)

External fragmentation solution — Compaction: Shuffle all occupied blocks to one end, combining all free holes into one large free block. Expensive — requires relocating all processes (possible only if relocation is dynamic at execution time).

6 Marks

Q2. Explain Paging — logical address structure, address translation, and page table with a numerical example.

Paging divides logical and physical memory into equal fixed-size blocks (pages/frames). The CPU generates a **logical address** consisting of two parts:

Logical address structure: [Page number (p) | Page offset (d)] p uses upper m-n bits; d uses lower n bits where page size = 2^n bytes, logical address space = 2^m bytes
Address translation: Physical address = (page_table[p] × page_size) + d

Numerical Example:

Logical address space = 32 bytes (5 bits), Page size = 4 bytes (2 bits for offset). So page number uses 3 bits.

Page #	Frame #	Physical location
0	5	Frames 5: physical 20-23
1	6	Frame 6: physical 24-27
2	1	Frame 1: physical 4-7
3	2	Frame 2: physical 8-11

Logical address 13 = binary 01101 Page number p = 011 = 3 Offset d = 01 = 1
page_table[3] = frame 2 Physical address = $2 \times 4 + 1 = 9$

TLB (Translation Lookaside Buffer): Hardware cache that stores recent page-to-frame translations. If page p is in TLB (TLB hit), frame number is obtained in 1 cycle without accessing page table in memory. TLB miss → access page table in memory (slower).

Effective Access Time (EAT) = hit_ratio × (TLB_time + mem_time) + (1-hit_ratio) × (TLB_time + 2×mem_time)
Example: hit=80%, TLB=20ns, mem=100ns → EAT = $0.8 \times 120 + 0.2 \times 220 = 96 + 44 = 140\text{ns}$

6 Marks

Q3. Explain Page Replacement Algorithms — FIFO, Optimal, LRU with a numerical example.

When a page fault occurs and no free frame exists, the OS must select a **victim page** to remove and load the new page. The algorithm determines which page to replace.

Reference string: 7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1 | Frames = 3

1. FIFO (First In First Out) — replace the page that has been in memory the longest:

When fault occurs: replace oldest loaded page (first in, first out) Total page faults = 15 (higher count) Note: Belady's anomaly – adding more frames can increase faults in FIFO

2. Optimal — replace page not used for longest time in the future (theoretical best):

Replace page whose next use is farthest in the future Total page faults = 9 (MINIMUM possible – theoretical optimum) Cannot implement in practice (requires future knowledge)

3. LRU (Least Recently Used) — replace page not used for the longest time in the past:

Replace page that was last used farthest in the past Total page faults = 12 (good practical approximation of Optimal) Implementation: counter (timestamp each access) or stack (most recently used at top)

Algorithm	Page Faults (3 frames)	Strategy	Practical?
FIFO	15	Replace oldest loaded page	Yes — easy to implement
Optimal	9 (minimum)	Replace page with farthest future use	No — requires future knowledge
LRU	12	Replace least recently used page	Approximate with hardware support

6 Marks

Q4. Explain Segmentation — concept, address translation, and compare with paging.

Segmentation is a memory management scheme that supports the user's view of memory. A program is divided into **variable-size segments** (logically distinct units) such as main program, function, array, stack, global variables.

Segmentation address structure: Logical address = Physical address = segment_table[s].base + d Condition: d < segment_table[s].limit (bounds checking)
If d >= limit → trap: addressing error

Segment Table: Each entry has a base (starting physical address) and limit (length of segment).

Comparison — Paging vs Segmentation:

Property	Paging	Segmentation
Division basis	Fixed-size pages (4KB)	Variable-size logical units
User visibility	Invisible — OS abstraction	Visible — programmer defines segments
Fragmentation	Internal only (last page)	External (variable sizes create holes)
Sharing	Share at page level	Easy to share entire segments (code, library)
Protection	Bits per page in page table	Per-segment (read-only code segment)
Example	Stack page 3, heap page 7	Stack segment, code segment, data segment

Segmentation with Paging: Segments are paged (each segment has its own page table). Gets advantages of both: logical segmentation (user view) + no external fragmentation (paging). Used in Intel x86 protected mode (80286+), Multics.

Unit III — File Systems

File Concepts | Access Methods | File System Structure | Allocation Methods | Directory | I/O Systems

2-MARK QUESTIONS — Unit III

2 Marks

Q1. Define a file. What are the attributes of a file?

A **file** is a named collection of related information stored on secondary storage (disk). It is the smallest logical unit of storage visible to users.

File attributes: Name (human-readable), Identifier (unique tag/inode number), Type (text, binary, executable), Location (pointer to device and block), Size (current size in bytes/blocks), Protection (access permissions rwx), Time/Date (creation, modification, last access), User identification (owner).

2 Marks

Q2. What are the three file access methods?

Method	Description	Example
Sequential access	Read/write bytes in order; can rewind	Tape drives, media files (MP3, video)
Direct (Random) access	Access any block directly by block number	Databases — "read block 100 directly"
Indexed access	Use an index of keys; look up key → pointer to record	Library catalog, database with B-tree index

2 Marks

Q3. Explain contiguous, linked, and indexed file allocation methods briefly.

Method	Description	Advantage	Disadvantage
Contiguous	All blocks in consecutive disk positions	Fast sequential access; simple	External fragmentation; file growth hard
Linked	Blocks scattered; each block has pointer to next	No external fragmentation; easy to extend	No random access; pointer overhead; fragile
Indexed	Index block stores pointers to all file blocks	Supports random access; no external fragmentation	Index block overhead; small files waste space

2 Marks

Q4. What is an inode in Unix/Linux file system?

An **inode (index node)** is a data structure in Unix/Linux that stores all metadata about a file EXCEPT its name. Each file has exactly one inode.

Inode contains: File type, size, owner UID/GID, permissions (rwxrwxrwx), timestamps (access/modify/change), hard link count, 12 direct block pointers, 1 indirect pointer, 1 double indirect pointer, 1 triple indirect pointer.

2 Marks

Q5. What is the difference between absolute path and relative path?

Property	Absolute Path	Relative Path
Definition	Full path from root directory	Path relative to current working directory

Starts with	/ (root) in Linux, C:\ in Windows	Current directory name or . (dot)
Example (Linux)	/home/student/os/notes.txt	os/notes.txt (if in /home/student)
Example (Windows)	C:\Users\Student\notes.txt	..notes.txt or ..\notes.txt

2 Marks

Q6. What is DMA (Direct Memory Access)?

DMA (Direct Memory Access) is a hardware mechanism that allows I/O devices to transfer data directly to/from main memory WITHOUT involving the CPU for each byte transfer.

How it works: CPU sets up DMA controller with source address (device buffer), destination address (memory), and count (bytes to transfer). CPU is free to do other work. DMA controller performs the transfer. When done, DMA interrupts CPU once to signal completion. Advantage: reduces CPU overhead from $O(n)$ per byte to $O(1)$ per transfer.

6-MARK QUESTIONS — Unit III

6 Marks

Q1. Explain File System Structure — layers and functions of each layer.

A file system is implemented as a multi-layered structure. Each layer uses services of the layer below it.

Layer 6: Application programs (user programs that open/read/write files) Layer 5: Logical file system (manages metadata – FCBs, directory structure, protection) Layer 4: File-organization module (logical to physical block translation) Layer 3: Basic file system (reads/writes physical disk blocks, buffer management) Layer 2: I/O control (device drivers, interrupt handlers) Layer 1: Devices (actual hardware: HDD, SSD, NVMe)

On-disk file system structures:

Structure	Location	Contents
Boot control block	First block of volume	Bootstrap loader to load OS
Volume control block	Second block (superblock)	Volume info: blocks, block size, free block/inode counts
Directory structure	Throughout volume	File names and their inode numbers
FCB (File Control Block)	inode table area	Per-file metadata: permissions, size, timestamps, block pointers
Free space map	Data blocks area	Bitmap or linked list of free blocks/inodes

File Operations and System Calls:

Operation	Description
create(name, permissions)	Allocate FCB, find space in directory, enter name
open(name, mode)	Search directory, load FCB to open-file table, return file descriptor
read(fd, buffer, n)	Use file position pointer, read n bytes from current block
write(fd, buffer, n)	Allocate new blocks if needed, write data, update FCB size

close(fd)	Remove from per-process table; decrement system-wide count; flush buffers
delete(name)	Remove directory entry, release FCB and all disk blocks, free space

6 Marks

Q2. Explain the three file allocation methods — Contiguous, Linked, and Indexed — with diagrams and comparison.

Contiguous Allocation — each file occupies a set of contiguous blocks on disk.

Directory entry: file "test.txt" | start=14 | length=3 Disk: [block 14: data1] [block 15: data2] [block 16: data3] Reading: simple — go to block 14, read 3 sequential blocks

- **Advantages:** Excellent sequential AND random access. Minimum head movement. Simple implementation.
- **Disadvantages:** External fragmentation — holes between files. Difficult to grow files. Need to know file size at creation time.

Linked Allocation — each block contains a pointer to the next block.

Directory entry: file "test.txt" | start=9 | end=25 Block 9: [data|ptr→16], Block 16: [data|ptr→1], Block 1: [data|ptr→10], Block 10: [data|ptr→-1] END pointer = -1 (null/EOF)

- **Advantages:** No external fragmentation. Files can grow easily. No pre-allocation needed.
- **Disadvantages:** No direct/random access (must traverse chain). Pointer overhead per block. Pointer corruption → whole file lost. FAT (File Allocation Table) is clustered linked allocation.

Indexed Allocation — one dedicated index block stores all pointers to data blocks.

Directory entry: file "test.txt" | index-block=19 Block 19 (index): [ptr→9, ptr→16, ptr→1, ptr→10, ...]
Block 9: data1, Block 16: data2, Block 1: data3, Block 10: data4

- **Advantages:** Supports direct access (jump to any block directly from index). No external fragmentation.
- **Disadvantages:** Index block overhead (wastes one full block even for tiny files). For very large files, one index block may not be enough → multilevel indexing or combined scheme (Unix inode).

Unix Combined Scheme (inode): 12 direct + 1 indirect (1024 ptrs) + 1 double indirect (1024²) + 1 triple indirect (1024³). With 4KB blocks and 4-byte pointers: max file size ≈ 4GB+ (double indirect alone handles 4GB).

6 Marks

Q3. Explain I/O System — hardware components, polling vs interrupts, and I/O scheduling.

The I/O system manages all communication between CPU and I/O devices.

I/O Hardware Components:

Component	Function	Example
I/O device	Physical hardware that performs I/O	HDD, SSD, keyboard, network card
Controller (adapter)	Electronics that control device operation	SATA controller, USB controller
Port	Connection point between controller and CPU bus	I/O port addresses
Bus	Shared communication path	PCIe, USB, SATA, NVMe

Device registers	Control, Status, Data-In, Data-Out registers	Read/write to control device
-------------------------	--	------------------------------

Polling vs Interrupts:

Method	How it works	CPU usage	Best for
Polling (busy wait)	CPU repeatedly checks device status register	Wastes CPU cycles while waiting	Fast devices; short waits; real-time
Interrupt	Device signals CPU when done; CPU continues other work	CPU free while device works	Slow devices; most I/O operations
DMA	Controller moves data directly; interrupts once when done	Minimal CPU involvement	Large block transfers (disk, network)

Disk Scheduling Algorithms: (order in which disk I/O requests are served)

- **FCFS:** Serve in arrival order. Simple but high seek time.
- **SSTF:** Serve nearest cylinder first. Reduces average seek. Risk of starvation for distant requests.
- **SCAN (Elevator):** Sweep in one direction serving all requests, reach end, reverse. Good throughput.
- **C-SCAN:** One direction only; wrap to start after reaching end. Uniform waiting times.
- **LOOK/C-LOOK:** SCAN/C-SCAN but stop at last request (not physical end). Best practical performance.

Unit IV — System Protection and Security

Protection Principles | Access Matrix | Cryptography | Program Threats | Authentication | Firewalling

2-MARK QUESTIONS — Unit IV

2 Marks

Q1. Differentiate between system protection and system security.

Aspect	Protection	Security
Concern	Controlling access of processes to resources	Defending system against external/internal attacks
Scope	Internal — between processes and OS resources	External + internal — attackers, malware, users
Mechanism	Access matrix, permissions, privilege rings	Encryption, authentication, firewalls, audit logs
Question asked	Is process P allowed to access resource R?	Is the claimed user actually who they say they are?
Example	File rwx permissions, dual mode	Password, HTTPS encryption, IDS

2 Marks

Q2. What is the principle of least privilege?

The **Principle of Least Privilege (POLP)** states that every program, user, or system process should have only the minimum access rights (permissions) necessary to perform its legitimate function — and nothing more.

Examples: A text editor should not have network access. A web server runs as "www-data" (not root). A student account cannot modify system files. Linux capabilities split root privilege into fine-grained parts (CAP_NET_BIND_SOCKET, CAP_SYS_TIME).

2 Marks

Q3. What is an access matrix? What are its rows and columns?

An **access matrix** is a general model for protection that represents the protection state of a system as a two-dimensional matrix.

Rows = Domains (subjects): users, processes, or groups that perform operations. **Columns** = Objects (resources): files, devices, memory, processes, other domains. **Entry** `access[i][j]` = set of access rights (read, write, execute, delete, etc.) that domain i is permitted on object j. Empty entry = no access allowed.

2 Marks

Q4. Define cryptography. What is symmetric encryption?

Cryptography is the science of transforming data into an unreadable form (ciphertext) using mathematical algorithms and keys, so that only authorised parties can read it.

Symmetric encryption uses the **same key** for both encryption and decryption. Also called secret-key or shared-key encryption. Fast but has a key distribution problem (how to securely share the key?).

Encryption: `plaintext + key → [algorithm] → ciphertext` Decryption: `ciphertext + key → [algorithm] → plaintext` Examples: AES (Advanced Encryption Standard), DES, 3DES, RC4

2 Marks

Q5. What is a Trojan horse? How does it differ from a virus?

Property	Trojan Horse	Virus
----------	--------------	-------

Definition	Legitimate-looking program that secretly performs harmful actions	Self-replicating code that attaches itself to other programs
Replication	Does NOT self-replicate	Replicates by attaching to files
User action	User installs it voluntarily	Virus runs when infected program runs
Example	Fake antivirus that installs malware	Boot sector virus, email attachment virus
Detection	Signature-based, behavior analysis	Antivirus signature scanning

2 Marks

Q6. What are the three factors of user authentication?

User authentication verifies the identity of a user. The three factors (something you **know**, **have**, **are**):

- **Knowledge factor — Something you KNOW:** Password, PIN, passphrase, security questions. Most common but weakest (can be guessed, stolen, or forgotten).
- **Possession factor — Something you HAVE:** Smart card, hardware token (RSA SecurID), mobile phone (OTP via SMS), USB key. Stronger — attacker needs physical object.
- **Biometric factor — Something you ARE:** Fingerprint, iris scan, face recognition, voice recognition, retinal scan. Strongest but expensive and can raise privacy concerns.

MFA (Multi-Factor Authentication) combines 2+ factors (e.g., password + OTP) for much stronger security. Used in banking, cloud services, VPNs.

6-MARK QUESTIONS — Unit IV

6 Marks

Q1. Explain Access Matrix implementation — Global Table, Access Control Lists, and Capability Lists with examples.

The access matrix defines who (domain) can do what (rights) to which resource (object). Since storing the full matrix is impractical for large systems, three implementation approaches exist:

Example Access Matrix:

Domain / Object	File F1	File F2	Printer	Process P1
D1 (Student)	read	read	print	—
D2 (Faculty)	read, write	read, write	print	wakeup
D3 (Admin)	all	all	all	all

1. Global Table — Single flat table of (domain, object, rights) triples:

Table rows: (D1,F1,read), (D1,F2,read), (D1,Printer,print), (D2,F1,read,write), ... Lookup: scan table for matching (domain, object) pair Disadvantage: Huge for large systems; slow linear search; cannot group common rights

2. Access Control Lists (ACLs) — Column decomposition (per-object list):

ACL for File F1: [(D1: read), (D2: read,write), (D3: all)] ACL for Printer: [(D1: print), (D2: print), (D3: all)]
Usage: D2 tries to write F1 → look up F1's ACL → find D2 entry → check write in {read,write} → ALLOW Advantage: Easy to know who can access an object. Easy revocation (remove from ACL).
Used in: Unix/Linux file permissions, Windows NTFS, AWS IAM

3. Capability Lists — Row decomposition (per-domain list of capabilities):

Capabilities of D2: [(F1: read,write), (F2: read,write), (Printer: print), (P1: wakeup)] A capability = unforgeable token granting specific rights to specific object Usage: D2 requests print → OS checks D2's capability list → find (Printer:print) → ALLOW Advantage: Easy to determine what a domain can do. Fast access check. Capabilities can be delegated. Disadvantage: Hard to revoke (may have been copied). Must protect capabilities from forgery. Used in: Mach OS, KeyKOS, modern microkernel systems

Property	ACLs	Capability Lists
Storage	Attached to each object	Attached to each domain/process
Easy to: find who can access object?	YES — read object's ACL	NO — scan all domains
Easy to: find what domain can do?	NO — scan all objects	YES — read domain's list
Revocation	Easy — modify object's ACL	Hard — must track all copies
Real-world example	Unix chmod, Windows NTFS	Mach capabilities, Kerberos tickets

6 Marks

Q2. Explain Program Threats — Malware types, and Security Defenses including Firewalling.

Program threats are malicious software (malware) that exploit vulnerabilities. Key types:

Threat	Definition	How it works	Example
Virus	Self-replicating code attached to legitimate programs	Runs when host program runs; copies itself to other files	ILOVEYOU, Melissa email virus
Worm	Self-replicating code that spreads via network	Exploits network vulnerabilities; no host file needed	Morris Worm, WannaCry ransomware
Trojan Horse	Legitimate-looking program with hidden malicious function	User installs voluntarily; secretly installs backdoor	Fake antivirus, game with keylogger
Logic Bomb	Dormant malware triggered by specific condition	Activates on date/event (e.g., fired employee revenge)	Sabotage payroll on specific date
Rootkit	Hides itself and other malware from OS	Replaces kernel system calls; invisible to standard tools	Sony BMG rootkit, Stuxnet
Ransomware	Encrypts user files; demands payment	Encrypts data; displays ransom note	WannaCry, CryptoLocker
Spyware	Monitors user activity covertly	Keystroke logging, screenshot, webcam access	Commercial tracking software

Security Defenses:

- **Antivirus/Anti-malware:** Signature-based (known patterns) + heuristic/behavior analysis (unknown threats)
- **Address Space Layout Randomization (ASLR):** Randomizes memory addresses → buffer overflow exploits fail
- **Data Execution Prevention (DEP/NX bit):** Marks stack/heap as non-executable → prevents code injection

- **Sandboxing:** Run programs in isolated environment (browser tabs, Docker containers)

Firewalling: A firewall is a network security device (hardware/software) that monitors and controls network traffic based on predetermined security rules.

Type	Layer	How it works	Example
Packet filter	Network (Layer 3)	Inspect IP/port headers — allow/block based on rules	iptables rules: block port 22 from external
Stateful inspection	Transport (Layer 4)	Track connection state; block unsolicited replies	Modern routers — track TCP sessions
Application proxy	Application (Layer 7)	Act as proxy; inspect application-level content	Web application firewall (WAF), Squid proxy
Next-gen firewall (NGFW)	All layers	Deep packet inspection, IDS, SSL inspection	Palo Alto, Cisco ASA

6 Marks

Q3. Explain Cryptography — Symmetric and Public Key Encryption with examples, and discuss digital signatures.

Cryptography secures data by making it unreadable without the correct key.

Symmetric Encryption (Secret Key):

Same key K used for both encryption and decryption: Encrypt: $C = E(K, P)$ — plaintext P encrypted with key K → ciphertext C Decrypt: $P = D(K, C)$ — ciphertext C decrypted with key K → plaintext P Algorithms: AES (128/256-bit), DES (56-bit, broken), 3DES, RC4 Key problem: How to securely share key K between sender and receiver?

- **Advantage:** Very fast — 100-1000x faster than public key. Suitable for bulk data encryption.
- **Disadvantage:** Key distribution problem. n users need $n(n-1)/2$ different keys for pairwise communication.

Asymmetric (Public Key) Encryption:

Key pair: Public key (shared openly) + Private key (secret) Encrypt with public key → only private key can decrypt Encrypt with private key → anyone with public key can decrypt (digital signature) Algorithms: RSA, ECC (Elliptic Curve), Diffie-Hellman Security basis: Computationally hard to factor large prime products (RSA) or compute discrete logarithms (ECC)

Alice → Bob secure communication: 1. Bob publishes his PUBLIC KEY to the world 2. Alice encrypts message with Bob's PUBLIC KEY → ciphertext 3. Ciphertext travels over internet 4. Only Bob decrypts with his PRIVATE KEY 5. Even if attacker intercepts ciphertext + knows public key → cannot decrypt

Digital Signatures — verify authenticity and integrity:

Signing: $\text{signature} = E(\text{sender_private_key}, \text{hash}(\text{message}))$ Verification: $\text{hash}(\text{message}) == D(\text{sender_public_key}, \text{signature})$ Provides: Authentication (only sender has private key) Integrity (hash verifies message not tampered) Non-repudiation (sender cannot deny signing)

In practice: Public key encryption for key exchange (slow) + symmetric encryption for bulk data. This is what HTTPS/TLS does — RSA/ECC to exchange AES session key, then AES for all data.

Unit V — Case Studies — Linux and Windows 7

Linux: Design, Kernel, Scheduling, Memory, File System, IPC | Windows 7: Architecture, Components, File Systems

2-MARK QUESTIONS — Unit V

2 Marks

Q1. State the design principles of Linux operating system.

Linux follows these key design principles (from Linus Torvalds, 1991):

- **Portability:** Written in C (not assembly). Runs on x86, ARM, MIPS, PowerPC, and hundreds of architectures.
- **Modularity:** Loadable kernel modules (device drivers, file systems loaded/unloaded at runtime without rebooting)
- **Open source:** GPL license — all source code freely available, modifiable, and redistributable
- **Multi-user and multitasking:** Hundreds of concurrent users; thousands of concurrent processes
- **Security:** DAC + MAC (SELinux/AppArmor), capabilities, namespaces, seccomp
- **POSIX compliance:** Compatible with Unix standards — portable application development

2 Marks

Q2. What is a Linux Kernel Module (LKM)?

A **Loadable Kernel Module (LKM)** is an object file containing code that can be dynamically inserted into the running Linux kernel without rebooting. It extends kernel functionality at runtime.

Uses: Device drivers (GPU, NIC, storage controllers), file system drivers (ext4, NTFS, FAT), network protocols (PPP, NDISWRAPPER), security modules (SELinux, AppArmor).

```
Commands: lsmod – list currently loaded modules insmod drv.ko – insert module into kernel
rmmod drv – remove module from kernel modprobe drv – load module with dependencies resolved
```

2 Marks

Q3. What are the design principles of Windows 7?

Windows 7 was designed with these principles:

- **Security and reliability:** UAC (User Account Control), BitLocker encryption, kernel patch protection (PatchGuard), code signing
- **Compatibility:** Backward compatible with Windows Vista/XP applications and hardware drivers
- **Performance:** Faster boot/shutdown, SuperFetch (prefetch frequently-used apps), ReadyBoost (USB as cache)
- **Layered architecture:** HAL (Hardware Abstraction Layer) + Kernel + Executive + Subsystems
- **NT kernel (monolithic-like):** Based on Windows NT architecture — executive services run in kernel mode for performance

2 Marks

Q4. What is IPC in Linux? Name the mechanisms.

Inter-Process Communication (IPC) in Linux provides mechanisms for processes to communicate and synchronize. Linux provides:

Mechanism	Description
Pipes ()	Unidirectional byte stream; connects stdout of one process to stdin of another

Named pipes (FIFO)	Persistent pipe with name in file system; unrelated processes can communicate
Shared memory	Fastest IPC — processes map same physical memory; needs synchronization
Message queues	Kernel-maintained queue; structured messages; survives process death
Sockets	Network IPC — AF_UNIX for local, AF_INET for network; used by client-server apps
Signals	Asynchronous notification (SIGKILL, SIGTERM, SIGCHLD, SIGUSR1)
Semaphores	POSIX and System V semaphores for synchronization

2 Marks

Q5. What is the Windows 7 HAL? What is its purpose?

The **Hardware Abstraction Layer (HAL)** is the lowest layer of Windows 7 architecture, sitting directly above the hardware. It hides hardware-specific details from the rest of the OS.

Purpose: (1) Portability — allows Windows NT kernel to run on different hardware (x86, x64, ARM, Itanium) with minimal changes to higher-level code. (2) Abstracts interrupt controllers, DMA, timers, and processor control registers. (3) Device drivers interact with HAL rather than directly with hardware — making drivers more portable.

2 Marks

Q6. Compare ext4 (Linux) and NTFS (Windows 7) file systems.

Feature	ext4 (Linux)	NTFS (Windows 7)
Max file size	16 TB	256 TB
Max volume size	1 EB	256 TB
Journaling	Yes (metadata + data)	Yes (metadata journaling)
Compression	No built-in	Yes — per-file NTFS compression
Encryption	No (use LUKS/eCryptfs)	Yes — EFS (Encrypting File System)
Permissions	POSIX rwxrwxrwx + ACLs	Windows ACLs — fine-grained per-user
Fragmentation	Less fragmentation (large pre-allocation)	More fragmentation; disk defrag needed

6-MARK QUESTIONS — Unit V

6 Marks

Q1. Explain the Linux kernel architecture — process management, scheduling, and memory management.

Linux is a **monolithic kernel** with loadable modules — all kernel services run in kernel space. The kernel consists of several major subsystems:

Linux Kernel Architecture (layered view):

```

User space: shell, user apps, system libraries (glibc) ████████████████████████████████████████
System call interface ████████████████████████████████████████ Kernel space: Process management |
Memory management | File systems Device drivers | Network stack | Security
(SELinux) ████████████████████████████████████████ Hardware Abstraction ████████████████████████████████████████
Hardware: CPU | RAM | Disk | NIC | GPU

```

Linux Process Management:

- Every process described by **task_struct** (PCB equivalent) — ~300 fields including PID, state, priority, parent pointer, file descriptors, signal handlers, memory map
- Process creation: **fork()** — copy-on-write (COW) pages shared until modified. **exec()** — replace process image. **clone()** — fine-grained sharing for threads
- States: TASK_RUNNING (R), TASK_INTERRUPTIBLE (S), TASK_UNINTERRUPTIBLE (D), TASK_STOPPED (T), TASK_ZOMBIE (Z)

Linux Scheduling — CFS (Completely Fair Scheduler, 2.6.23+):

CFS uses virtual runtime (vruntime) – tracks how long each process has run vruntime increases proportional to actual time / weight (nice value) Always run process with LOWEST vruntime – ensures fairness Red-black tree (sorted by vruntime) – leftmost node = next to run $O(\log n)$ schedule + $O(1)$ enqueue/dequeue Nice value: -20 (highest priority) to +19 (lowest) Weight: $weight = 1024 / (1.25^{nice})$ (nice=0 → weight=1024)

Linux Memory Management:

- **Virtual memory:** Each process has its own 4GB (32-bit) or 128TB (64-bit) virtual address space split: 3GB user space + 1GB kernel (32-bit)
- **Physical pages:** Managed by buddy system (power-of-2 page blocks) for efficient allocation
- **Slab allocator:** Efficient small object allocation for kernel data structures (dentries, inodes)
- **Swap:** /swap partition or swapfile; managed by kswapd daemon; LRU-based page reclaim
- **OOM killer:** When memory critically low, kernel selects and kills process with highest "badness" score

6 Marks

Q2. Explain the Linux File System — VFS, ext4 structure, and I/O in Linux.

VFS (Virtual File System) is an abstraction layer in the Linux kernel that provides a uniform interface to different file system types (ext4, NTFS, FAT, NFS, proc, sysfs, tmpfs).

VFS layer: User calls `open("/home/user/file.txt")` → VFS finds filesystem type → calls `ext4_open()` → ext4 reads inode from disk → returns file descriptor VFS provides 4 object types: 1. superblock – represents entire mounted filesystem 2. inode – represents a file/directory (metadata) 3. dentry – directory entry (name → inode mapping) 4. file – represents open file instance per-process

ext4 File System Structure:

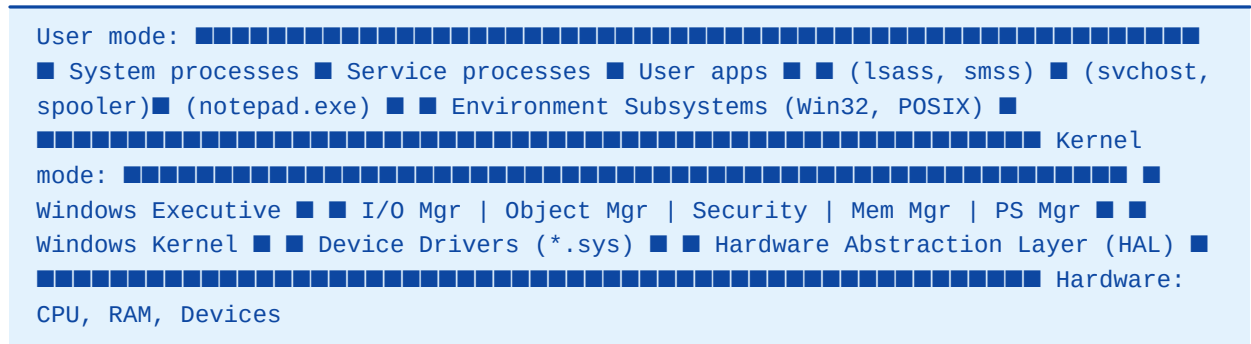
Structure	Location	Contents
Superblock	Block group 0, offset 1024	Total inodes, blocks, block size, fs state, magic number (0xEF53)
Block group descriptors	After superblock	For each group: inode table location, block/inode bitmaps location, free counts
Inode bitmap	Per block group	1 bit per inode — used (1) or free (0)
Block bitmap	Per block group	1 bit per block — used (1) or free (0)
Inode table	Per block group	Array of inodes (128 or 256 bytes each)
Data blocks	Remaining blocks	Actual file data, directory entries

Linux I/O Architecture:

- Everything is a file in Linux: regular files, devices (`/dev/sda`), pipes, sockets, `/proc`, `/sys`

- 6 Marks** Q3. Explain Windows 7 Architecture — Executive, Kernel, Subsystems, and key components.

Layered Architecture:



Component	Function
I/O Manager	Manages all I/O requests; provides IRP (I/O Request Packets) framework for drivers
Object Manager	Creates and manages all kernel objects (processes, threads, files, events, mutexes)
Security Reference Monitor	Enforces access control; checks ACLs; generates audit events; implements UAC
Memory Manager	Virtual memory; page tables; working set management; section objects (shared memory)
Process/Thread Manager	Creates/terminates processes and threads; manages scheduling with kernel dispatcher
Plug & Play Manager	Detects new hardware; loads appropriate device drivers automatically
Power Manager	Manages sleep/hibernate states; ACPI power management; device power states
Registry	Hierarchical database for OS and application configuration (HKLM, HKCU)

Windows Networking: Windows 7 uses a layered networking model: WinSock API → AFD driver → TDI transport → TCP/IP stack (tcpip.sys) → NDIS → Network adapter driver. Supports: SMB (file sharing), NetBIOS, WPS, HomeGroup, Active Directory integration, IPv4/IPv6, 802.11 wireless.

OS PC 505 AI — Complete 2-Mark & 6-Mark Answers | All 5 Units | BE AIML — Osmania University | Page 22

API Category	Key Functions
Process/Thread	CreateProcess(), CreateThread(), TerminateProcess(), WaitForSingleObject()
Memory	VirtualAlloc(), VirtualFree(), MapViewOfFile(), HeapAlloc()
File I/O	CreateFile(), ReadFile(), WriteFile(), CloseHandle(), FindFirstFile()
Registry	RegOpenKey(), RegQueryValue(), RegSetValue(), RegDeleteKey()
GUI	CreateWindow(), SendMessage(), GetMessage(), TranslateMessage()
Networking	WSASocket(), connect(), send(), recv(), WSASocket() — WinSock API

Quick Reference — All Formulas & Definitions

Formula / Concept	Expression / Definition
CPU Turnaround Time	$TAT = \text{Completion Time} - \text{Arrival Time}$
CPU Waiting Time	$WT = TAT - \text{Burst Time}$
CPU Utilization (%)	$(\text{Busy time} / \text{Total time}) \times 100$
Avg Access Time with TLB	$EAT = \alpha(\text{TLB_time} + \text{mem}) + (1-\alpha)(\text{TLB_time} + 2 \times \text{mem})$; $\alpha = \text{hit ratio}$
Page table address translation	$\text{Physical} = \text{page_table}[\text{page\#}] \times \text{page_size} + \text{offset}$
Need (Banker's)	$\text{Need}[i] = \text{Max}[i] - \text{Allocation}[i]$
Available resources	$\text{Available} = \text{Total} - \text{SUM}(\text{Allocation for all processes})$
Disk head movement	SUM of $ \text{cylinder_next} - \text{cylinder_current} $ for all steps
Logical address breakdown	Upper bits = page/segment number; lower bits = offset
File max size (Unix inode)	12 direct + 1024 indirect + 1024^2 double + 1024^3 triple blocks
Semaphore wait(S)	while $S \leq 0$ do nothing; $S = S - 1$ (atomic)
Semaphore signal(S)	$S = S + 1$ (atomic)
Process states	New $\rightarrow$ Ready $\rightarrow$ Running $\rightarrow$ Waiting $\rightarrow$ Terminated
POSIX thread create	<code>pthread_create(&tid, NULL, func, arg); pthread_join(tid, NULL)</code>
Linux nice range	Nice: -20 (highest priority) to +19 (lowest priority)
CFS scheduling	Always run process with lowest vruntime; tracked in red-black tree
EAT (demand paging)	$EAT = (1-p) \times \text{memory_time} + p \times \text{page_fault_service_time}$; $p = \text{fault rate}$
Symmetric encryption	Encrypt+Decrypt use same key K; Examples: AES, DES, 3DES
Public key encryption	Encrypt with public key $\rightarrow$ decrypt with private key; Examples: RSA, ECC
Digital signature	$\text{sign} = E(\text{priv_key}, \text{hash}(\text{msg}))$; verify: $\text{hash}(\text{msg}) == D(\text{pub_key}, \text{sign})$